

RNG com não repetição (3 Bits)

ESPECIFICAÇÃO

Projeto do Bloco Digital

Matheus Grossi

Data: 20/01/2026
Rev.: 1

Histórico de Versões

A tabela abaixo registra o histórico de alterações deste documento:

Versão	Data	Autor	Descrição das Mudanças
1.0	20/01/2026	Matheus Grossi	Versão inicial (RNG - 3 bits).

Siglas e Acrônimos

RNG	Random Number Generator (Gerador de Números Aleatórios)
PRNG	Pseudo-Random Number Generator
ASIC	Application Specific Integrated Circuit
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine (Máquina de Estados Finita)
HDL	Hardware Description Language
RTL	Register Transfer Level

1 Visão Geral e Objetivos

Este projeto consiste na implementação em Verilog de um Gerador de Números Pseudoaleatórios (PRNG). O sistema foi projetado para gerar números de **3 bits** (faixa de 0 a 7) utilizando 4 sementes (*seeds*) distintas baseadas em tabelas de permutação.

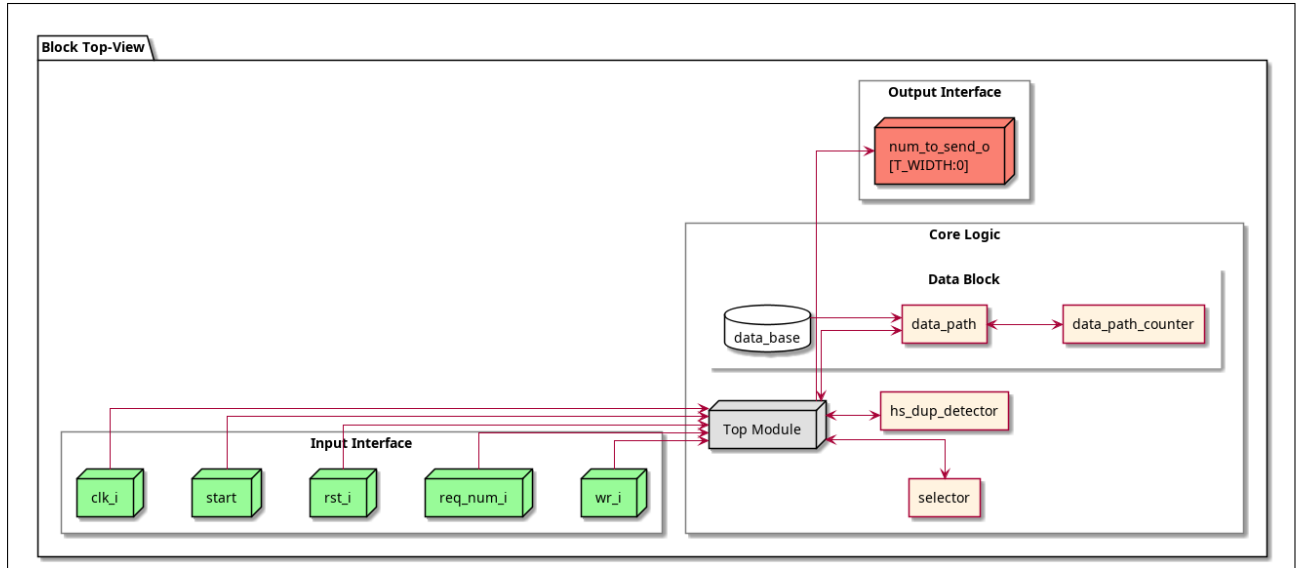


Figure 1: Vista de Topo do Sistema (Topview)

A principal característica deste projeto é o mecanismo de **detecção de duplicatas**, que assegura que os números gerados não se repitam dentro de um ciclo de operação, garantindo a unicidade da saída até que a sequência seja reiniciada ou o buffer seja limpo.

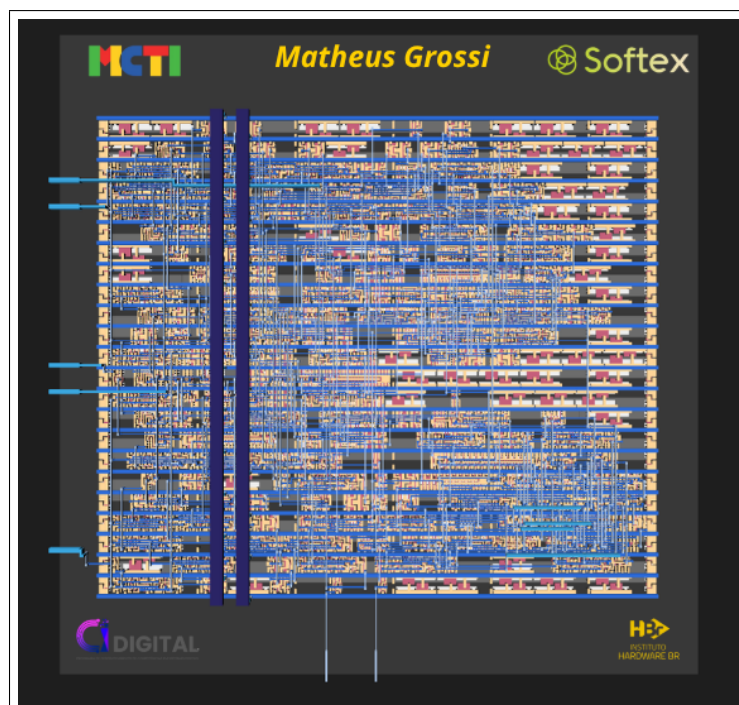


Figure 2: Vista Pós-Síntese do SoC

2 Identificação do Projeto (RTL Design)

Este design é implementado utilizando HDL padrão de indústria e estruturado para facilitar a integração e síntese.

- **Nome do bloco:** RNG Não Repetitivo (3 bits)
- **Linguagem de Descrição:** Verilog
- **Tecnologias Alvo:** FPGA e ASIC
- **Características Chave:** Interface de Handshake, 4 Seeds, Histórico de não-repetição.

Abaixo estão as vistas explodidas do design após síntese em FPGA:

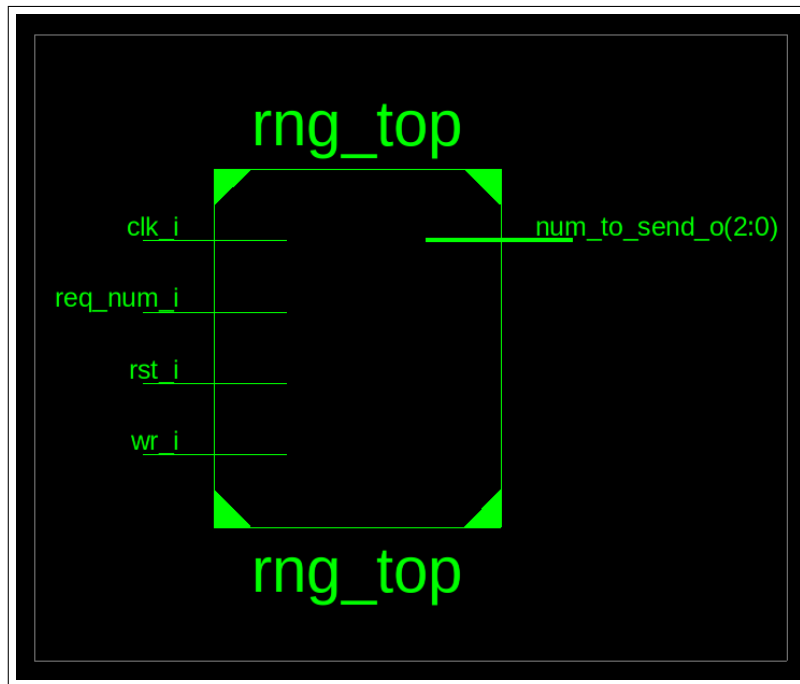


Figure 3: Visão de Topo do Circuito (Síntese)

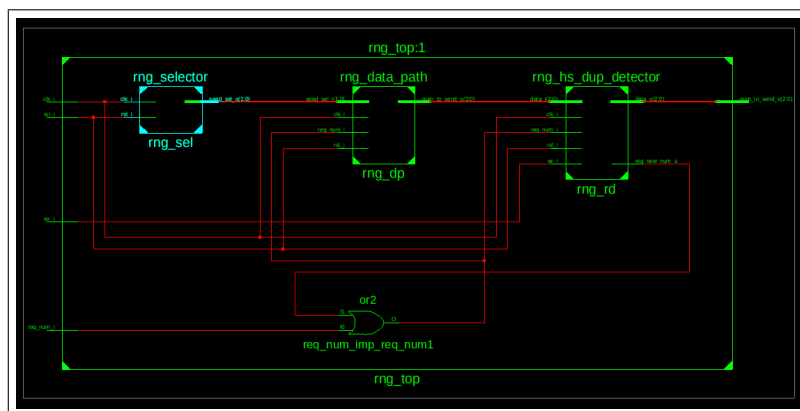


Figure 4: Vista Explodida 1

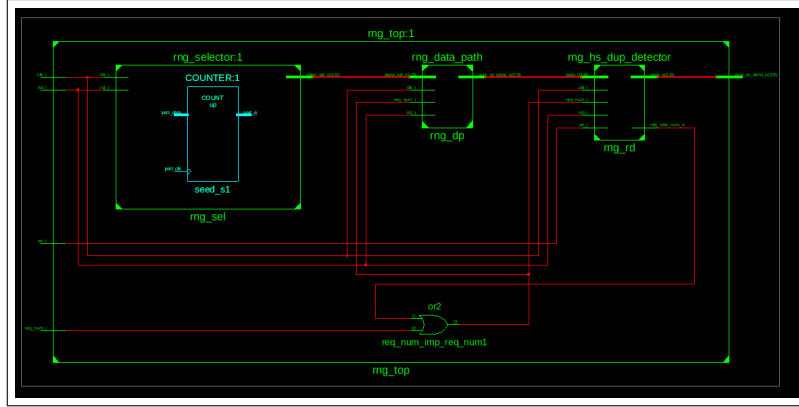


Figure 5: Vista Explodida 2

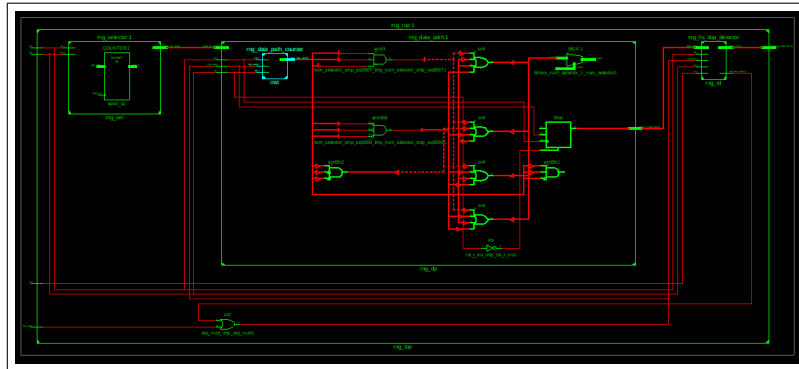


Figure 6: Vista Explodida 3

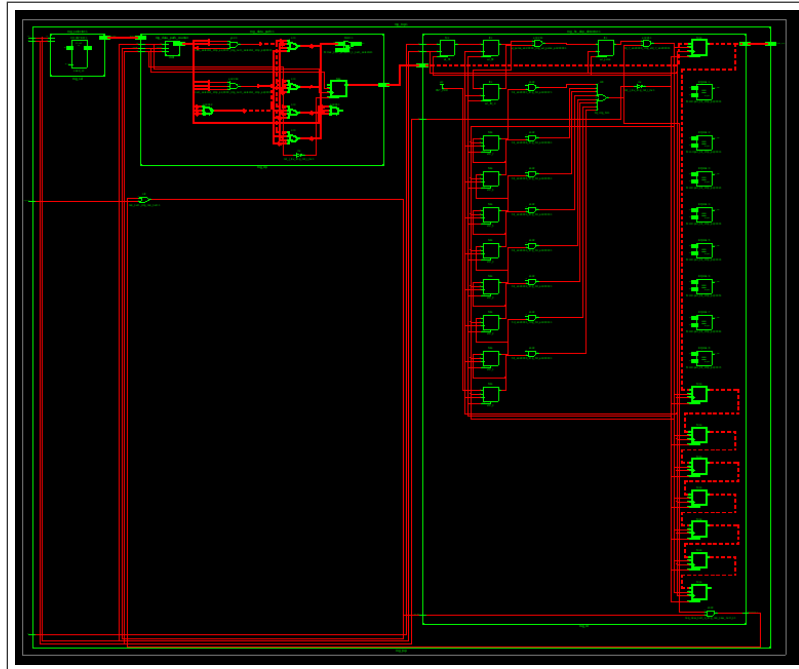


Figure 7: Vista Explodida 4

3 Fluxo de Funcionamento e Timing

O bloco opera através de um fluxo coordenado para garantir a validade dos dados:

1. **Reset:** O sistema é resetado (`rst_i`), limpando a memória do detector de duplicatas.
2. **Seleção de Seed:** O módulo `rng_selector` utiliza um contador *free-running* para definir qual das 4 tabelas será usada.
3. **Geração e Validação:** O `rng_data_path` gera um candidato. O `rng_hs_dup_detector` verifica duplicatas. Se existir, descarta e busca o próximo. Se for único, disponibiliza na saída.
4. **Confirmação (Handshake):** O sinal `wr_i` confirma a leitura e salva o valor no histórico.

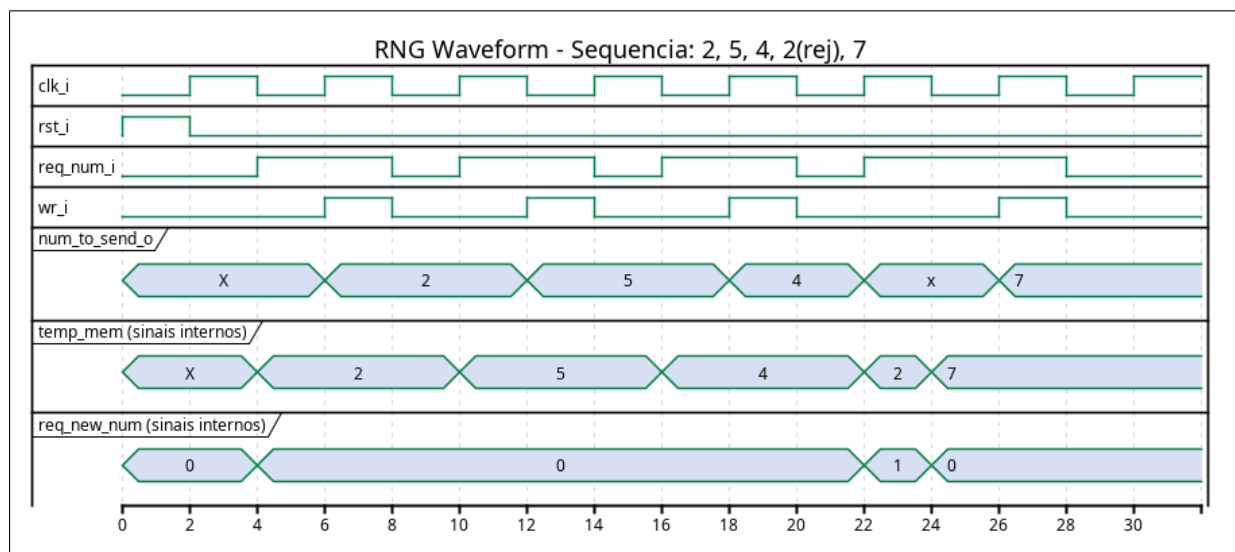


Figure 8: Diagrama de Temporização (Timing)

4 Parâmetros de Configuração

O sistema utiliza parâmetros para cálculo automático de larguras de barramento e profundidade de memória.

Parâmetro	Padrão	Descrição
DEPTH	72	Profundidade base para cálculos de dimensão.
T_DEPTH	DEPTH-1	Profundidade real (zero-based).
WIDTH	3	Largura do número gerado (0 a 7).
T_WIDTH	WIDTH-1	Largura real do vetor de dados.
SEED_TOT_NUMB	12	Total de números agrupados nas sementes.
COUNT_WIDTH	$\lceil \log_2(\dots) \rceil$	Largura do contador de ciclos (4 bits).

5 Visão Geral do Sistema

5.1 Interfaces do Bloco

A tabela abaixo descreve os sinais do módulo principal `rng_top`.

Nome	Dir	Bits	Descrição Funcional
<code>clk_i</code>	I	1	Clock do sistema.
<code>rst_i</code>	I	1	Reset assíncrono (Active Low).
<code>req_num_i</code>	I	1	Requisição de novo número aleatório.
<code>wr_i</code>	I	1	Sinal de escrita/confirmação. Salva no histórico de não-repetição.
<code>num_to_send_o</code>	O	3	Número pseudoaleatório válido gerado.

Table 1: Definição de Interfaces (I/O)

5.2 Estrutura de Sub-blocos

O projeto está organizado nos seguintes módulos funcionais:

- **`rng_top`:** Módulo topo que integra controle, dados e detector.
- **`rng_data_path`:** Seleciona o valor numérico via multiplexação baseada na seed e índice.
- **`rng_selector`:** Gera o sinal de seleção de seed (2 bits) de forma pseudoaleatória.
- **`rng_hs_dup_detector`:** Memória de histórico e lógica de comparação. Rejeita números repetidos e armazena novos valores válidos mediante `wr_i`.
- **`rng_data_path_counter`:** Contador cíclico de 3 bits para percorrer as tabelas.

6 Verificação e Simulação

Os ambientes de teste estão localizados em `testbenchs` e utilizam scripts de automação (Makefiles/Bash) para geração de ondas. O waveform abaixo demonstra o comportamento real obtido em simulação.

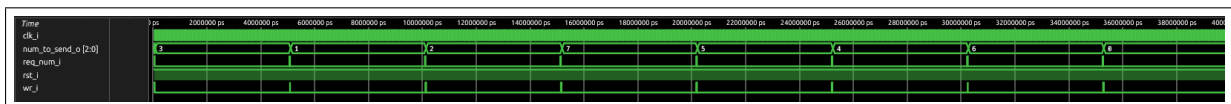


Figure 9: Waveform da Simulação

Metodologia: Design Síncrono com validação via Handshake e Testbenchs Automatizados.